NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

Centre for Professional and Continuing Education

(SCTP) Advanced Professional Certificate

Data Science and AI



Interactive site: https://su-ntu-ctp.github.io/6m-data-3.8-Computer-Vision/

L08 Computer **Vision**

CNNs & Transfer Learning · Sarah auto-tags product photos

Format: ~90 min slides · ~90 min hands-on · self-paced extensions

Copyright © 2026 Nanyang Technological University. All rights reserved.

Anton Thai | DS-6 — 17/10/26, 5:05 PM
Hi @Shumin | Instructor and Team, could you please advise on the CV package for extracting number, date, text from industrial machine HMI screen (like Siemens HMI) and from an equipment indication meter (gauge/ manometer)?
In some cases, the screen layouts and text fields are fixed, just need to capture the changes in digits or the pointer?
Thanks

FF | Instructor [LABS], — 17/10/26, 7:17 PM
For extracting data from both digital HMI screens and analog gauges, you are dealing with two distinct computer vision tasks: Optical Character Recognition (OCR) for the text/digits, and geometric/angle detection for the pointers.

Here are the best packages and approaches for your use cases:

Digital HMI Screens (Text, Dates, Numbers)Since you mentioned the screen layouts are often fixed, you don't need heavy deep learning to find the text—you just need to read it.

OpenCV (cv2): Essential for preprocessing. Use it to define static Regions of Interest (ROIs) based on pixel coordinates. Crop these specific bounding boxes, convert them to grayscale, and apply thresholding (binarization) to make the text pop against the background.

PaddleOCR: Currently one of the most robust, fast, and accurate open-source OCR libraries. It handles varying fonts and industrial display types better than older tools.

EasyOCR: A great lightweight alternative based on PyTorch that is very easy to implement out-of-the-box.

Tesseract (pytesseract): The traditional standard. It works exceptionally well only if the image is heavily preprocessed (cropped, high contrast, no background noise).

Workflow: OpenCV (Crop fixed ROI -> Grayscale -> Threshold) -> PaddleOCR (Read text string).
Analog Meters and Gauges (Pointers)
Extracting values from analog gauges is purely geometric. You need to find the angle of the pointer relative to the scale.

OpenCV (Traditional Approach): You can solve this entirely without machine learning.

Use Hough Circle Transform to detect the dial's face.

Use color masking or thresholding to isolate the pointer needle.

Use Hough Line Transform or contour detection to find the line of the needle.

Calculate the angle of the line and map it to your minimum/maximum gauge values.

Ultralytics YOLO (Modern/Robust Approach): If lighting conditions change or cameras move slightly, traditional OpenCV might fail. You can train a lightweight YOLOv8 Keypoint Detection model.

Label three keypoints: the center of the gauge, the minimum value mark, and the tip of the pointer.

Calculate the angle using the coordinates of these keypoints.

Recommendation: Start with OpenCV to hardcode the ROIs for the HMI screens and pass them to PaddleOCR. For the analog gauges, try the OpenCV Hough Transform method first; if it is too brittle against reflections on the gauge glass, pivot to training a small YOLO pose/keypoint model.

Boon Wee | DS6 — 17/10/26, 9:11 PM
Anton. You should use opc interfaces to get the data from HMI or field devices. Siemens have such products to integrate with field devices. There are historians that can collect data from field devices

Shumin | Instructor
Hi @Anton Thai | DS-6 ,
Both are good suggestions!

I would recommend checking whether the data can be extracted directly from the machine via OPC UA or other industrial communication protocols first. This would generally be more reliable and accurate than using CV, especially for real-time monitoring.
However, if direct data access isn't possible (e.g. legacy equipment, restricted access), then CV is definitely a viable alternative. Since you mentioned that the layouts are fixed, you can simplify the problem quite significantly:

Digital HMI: Use OpenCV to crop the specific regions of interest (ROI), followed by OCR libraries such as PaddleOCR or EasyOCR to extract the numbers/text.
Analog gauges: You can explore OpenCV techniques such as Hough Line Transform to detect the pointer angle and map it to the corresponding gauge reading.
For fixed layouts, you may not even need to train a deep learning model! Traditional CV techniques could be sufficient. One additional consideration would be the image capture conditions (lighting, reflections, camera angle), as these can affect the accuracy of the readings.

Where We Left Off

From session features to pixels

Marcus, end of L07:

"Can we auto-tag product photos as dress, shirt, sneaker?
We have ~10,000 new uploads each season."

What's different this week

- **New modality** — Inputs are images (28×28×1 or 224×224×3 tensors), not tabular rows
- **New architecture** — Convolutional layers replace the flat MLP first layer
- **New tool: transfer** — Pretrained models from torchvision — don't start from scratch
- **Same training loop** — zero_grad → forward → loss → backward → step (L07 unchanged)

Copyright © 2026 Nanyang Technological University. All rights reserved.





Copyright © 2026 Nanyang Technological University. All rights reserved.

Where have you seen CV in your day to day?

Copyright © 2026 Nanyang Technological University. All rights reserved.

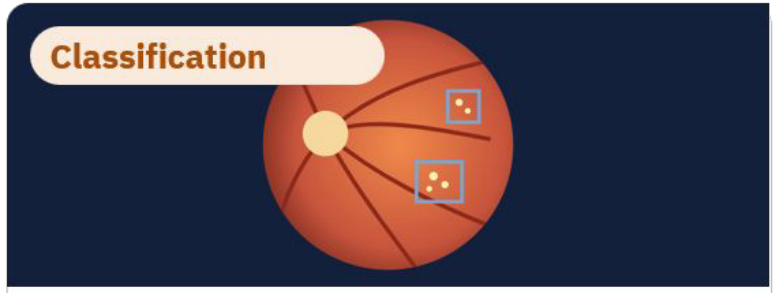
Where computer vision works today



Tesla · self-driving

8 cameras give a 360° view, with no lidar. Since FSD v12 (2024), one network maps video straight to steering and braking.

End-to-end neural network: video in, controls out



Google · eye screening

ARDA checks retina photos for diabetic eye disease, a leading cause of blindness. 1M+ patients in India, Thailand, Australia.

CNN classifier (Inception-v3 in the 2016 study)



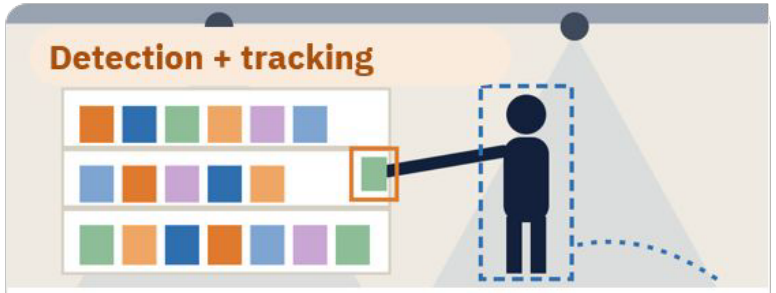
BMW · factory inspection

AIQX cameras photograph every car on the line and flag a wrong badge or missing part before it moves on. In all BMW plants.

Image classification and anomaly detection

Copyright © 2026 Nanyang Technological University. All rights reserved.

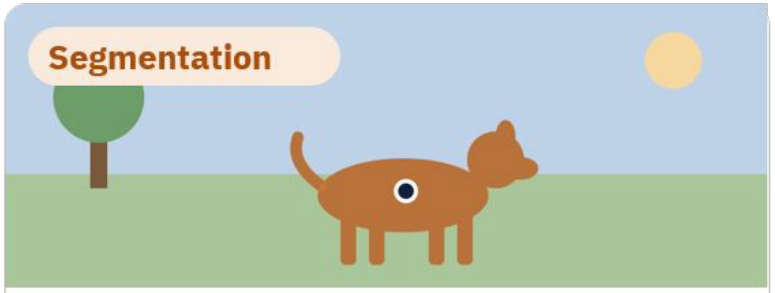
...and where you meet it every day



Amazon · Just Walk Out

Ceiling cameras and weighing shelves track what each shopper picks up, then charge on exit. About 130 partner sites.

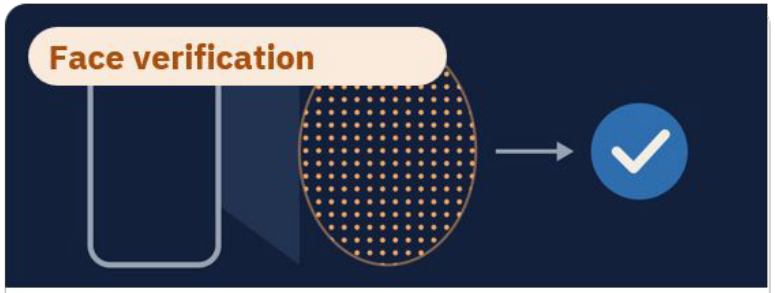
Multi-camera detection, tracking and sensor fusion



Meta · Segment Anything

Type "dog" or click it, and SAM 3 outlines it pixel by pixel, then follows it through a video. Released openly, Nov 2025.

Promptable segmentation model



Apple · Face ID

Projects 30,000 infrared dots to map your face in 3D, then checks it against your saved face. It never leaves the phone.

Neural network on the phone's own chip

Copyright © 2026 Nanyang Technological University. All rights reserved.

Today's Three Questions

From pixels to predictions

1

What does an image LOOK LIKE to a neural network?

A tensor of pixel intensities — and why MLPs choke on it.

2

What IS a convolutional neural network?

Stack of (Conv → ReLU → Pool) blocks + a classifier head.

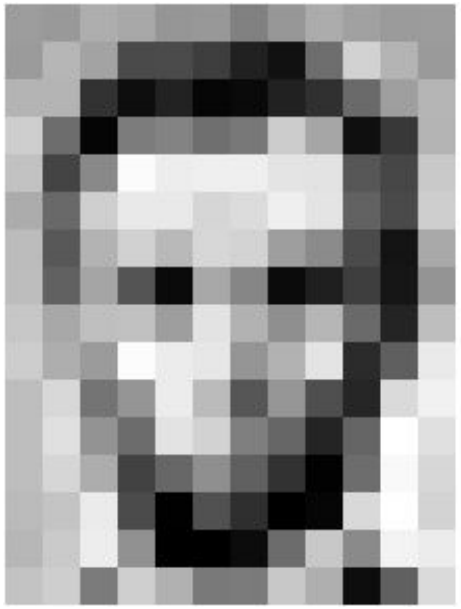
3

When should we reach for transfer learning?

When data is small AND the domain is close to ImageNet.

Copyright © 2026 Nanyang Technological University. All rights reserved.

An Image Is Just A Tensor



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218



https://setosa.io/ev/image-kernels/

Copyright © 2026 Nanyang Technological University. All rights reserved.

An Image Is Just A Tensor

Numbers in a particular shape

Grayscale 28×28 image

```
shape = (1, 28, 28)
```

Colour 224×224 photo

```
shape = (3, 224, 224)
```

PyTorch batch convention

```
(B, C, H, W)
```

Pixel values

Raw: 0-255 (uint8)
Normalised: 0.0-1.0 (float)

Grayscale: 1 intensity per pixel
RGB: 3 intensities per pixel (R, G, B)

To a neural network this is literally a NumPy array. No special "image" magic.

Copyright © 2026 Nanyang Technological University. All rights reserved.

The Kernel Intuition

Nine numbers that detect edges

An edge-detection kernel

```
-1   -1   -1
-1    8   -1
-1   -1   -1
```

Flat region: 8 - 8 = 0 → output near 0
Edge: large difference → big output

Nine numbers detect ALL edges everywhere in the image.

Parameter savings

`Conv2d(3, 32, kernel=3)`

```
32 kernels × (3×3×3) + 32
  = 864 + 32 = 896 params
```

MLP first layer, same input/output

```
224×224×3 × 32 = 4,816,896 params
```

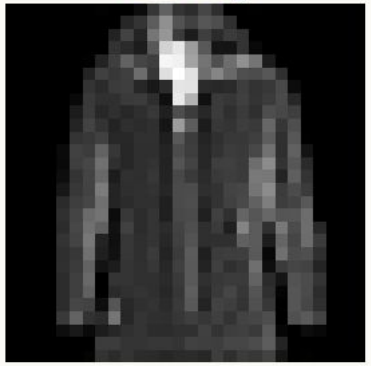
~5,000× fewer parameters

Copyright © 2026 Nanyang Technological University. All rights reserved.

Same coat, different kernels

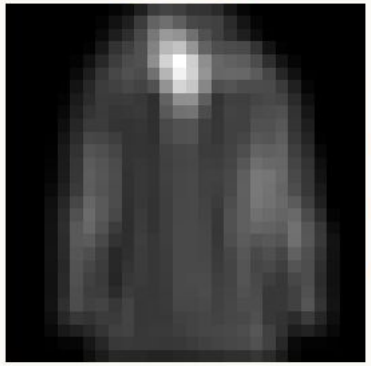
Weights summing to 1 keep brightness (blur, sharpen). Weights summing to 0 keep only changes (edges).

Identity



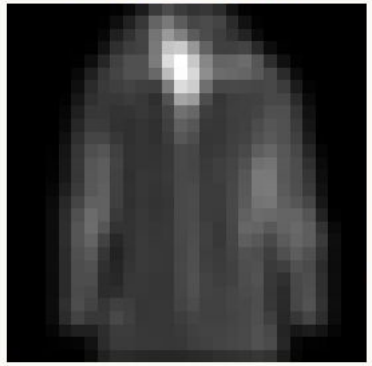
0	0	0
0	1	0
0	0	0

Box blur



1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Gaussian blur



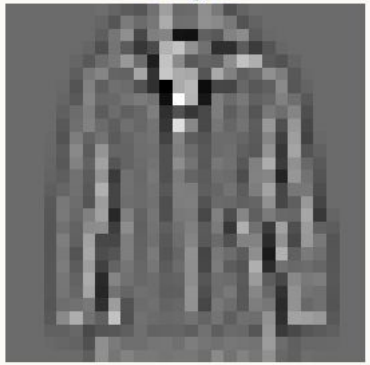
1/16	2/16	1/16
2/16	4/16	2/16
1/16	2/16	1/16

Sharpen



0	−1	0
−1	5	−1
0	−1	0

Edge



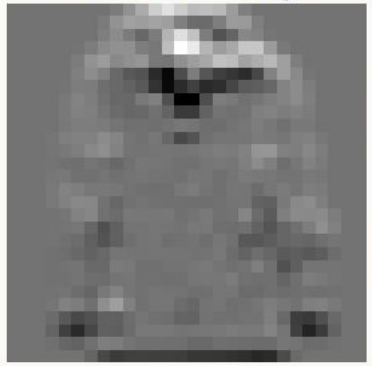
−1	−1	−1
−1	8	−1
−1	−1	−1

Vertical edges



−1	0	1
−2	0	2
−1	0	1

Horizontal edges



−1	−2	−1
0	0	0
1	2	1

Emboss



−2	−1	0
−1	1	1
0	1	2

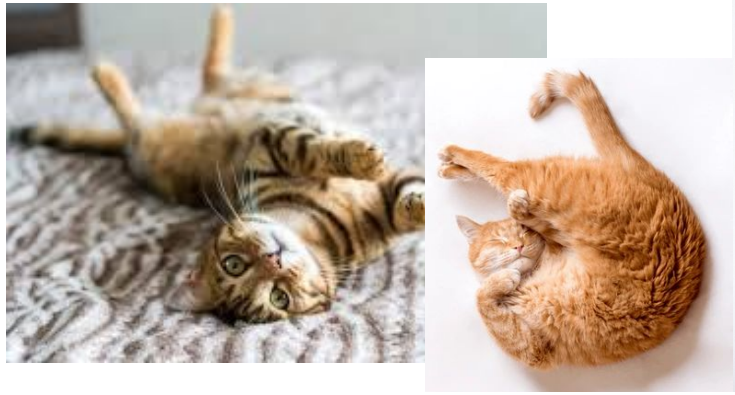
Copyright © 2026 Nanyang Technological University. All rights reserved.

How does AI classify an image?



Copyright © 2026 Nanyang Technological University. All rights reserved.

Potential Problems?



Distortion



Occlusion



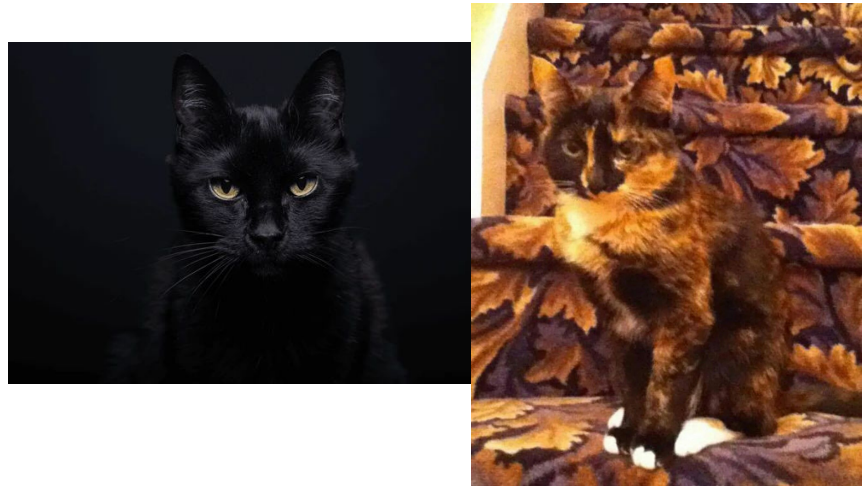
Viewpoint variation



Illumination



Scale

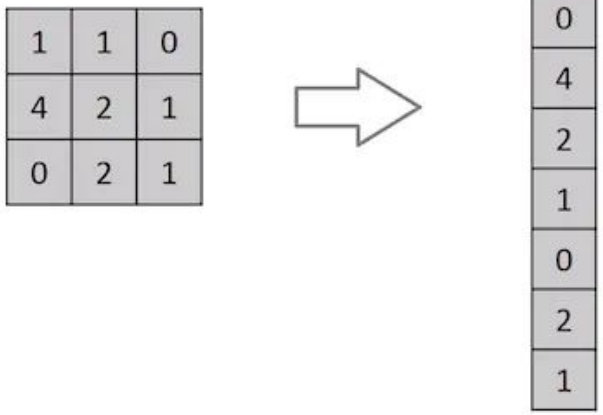


Background Noise

Copyright © 2026 Nanyang Technological University. All rights reserved.

Why CNNs?

CNN
Or
Feedforward
Network?



Considerations



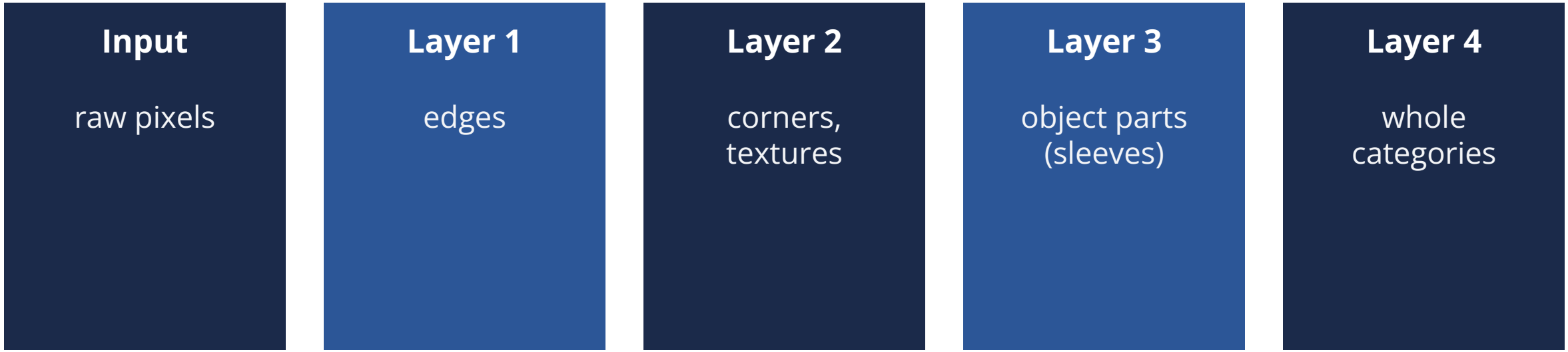
Compute | Generalization | Correlations

Copyright © 2026 Nanyang Technological University. All rights reserved.

Stacking = Compositionality

Why deep CNNs work

Each layer combines features from the layer below — hierarchy of visual concepts

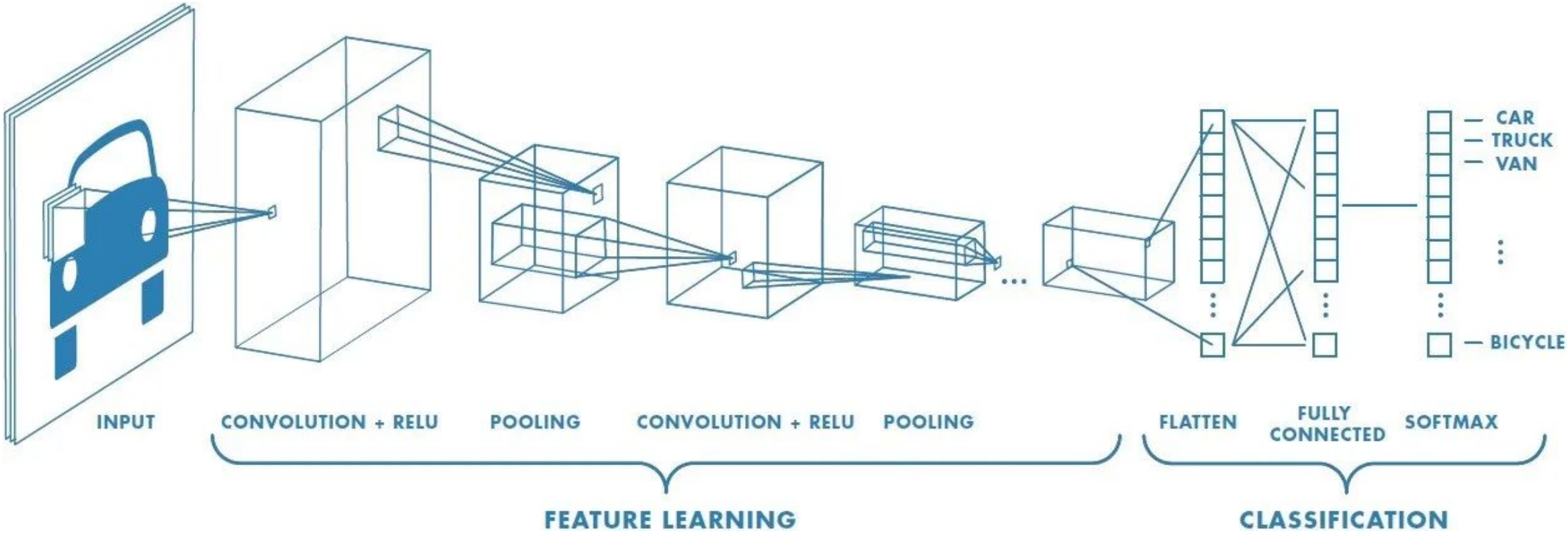


Each kernel is small (e.g. 3×3), but stacking 4-5 layers gives a large effective receptive field — deep neurons see large chunks of the image and respond to complex compositions of simple parts.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Convolutional Neural Networks (CNNs)

CNNs are a class of deep neural networks, specifically designed for Computer Vision tasks. They have been highly successful in various computer vision tasks and are the backbone of many state-of-the-art computer vision applications.

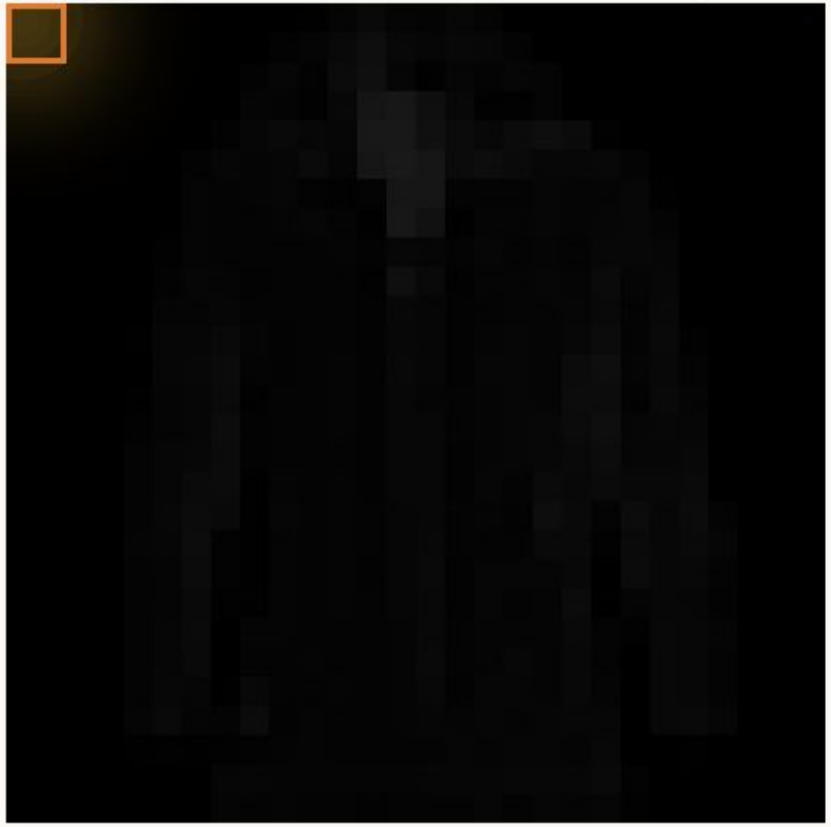


https://poloclub.github.io/cnn-explainer/

Copyright © 2026 Nanyang Technological University. All rights reserved.

How a CNN looks at an image: one small patch at a time

Image in the dark, torch on a 3×3 patch



What the torch sees

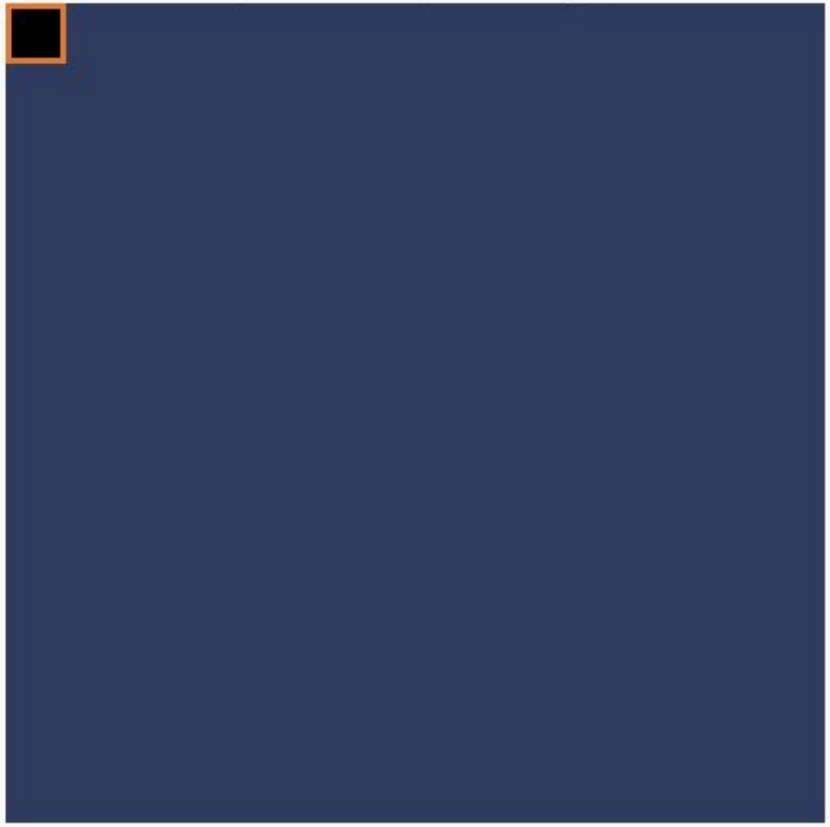
0.00	0.00	0.00
0.00	0.00	0.00
0.00	0.00	0.00

× edge kernel, then sum

= 0.00

one pixel of the feature map

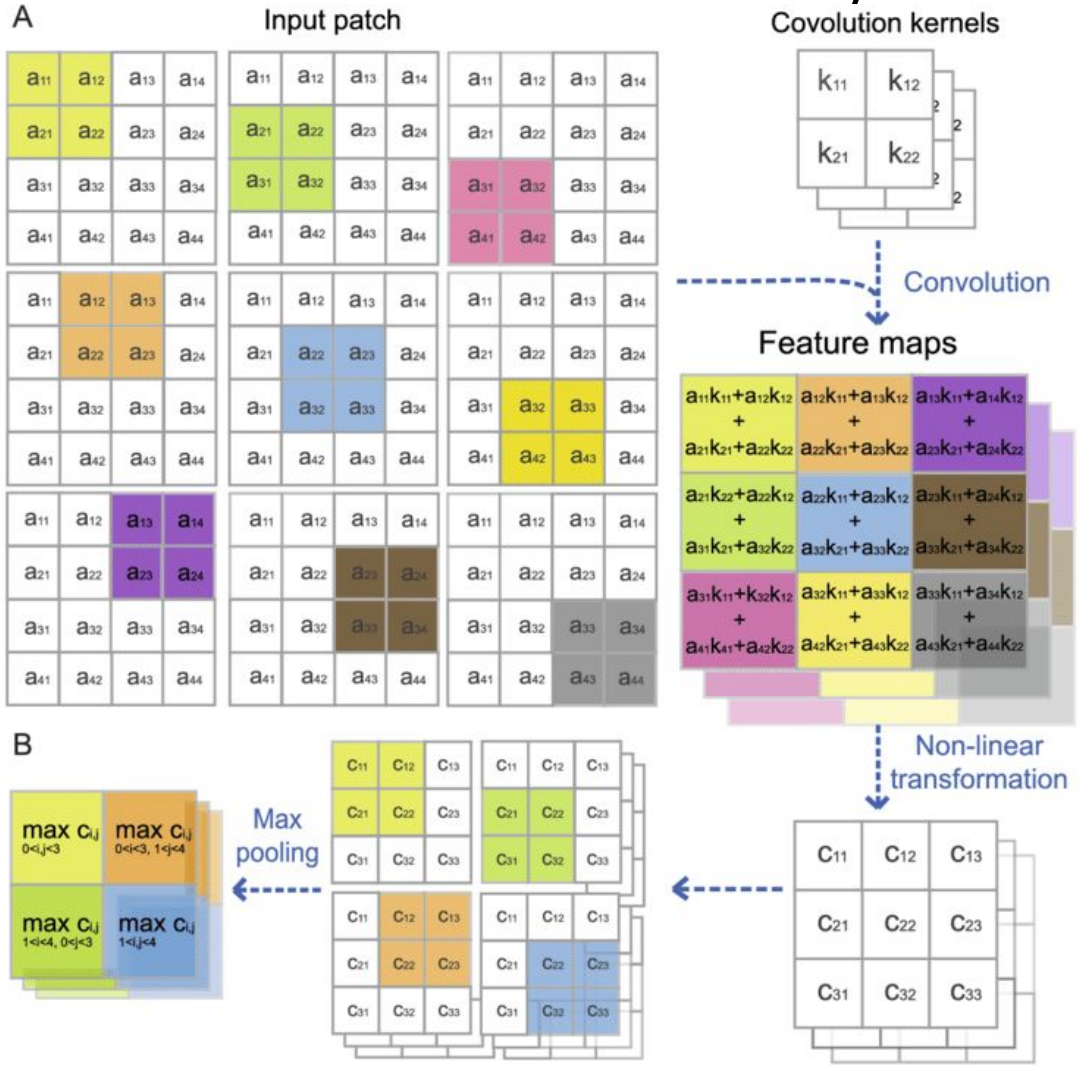
Feature map fills in, patch by patch



Same 9 weights at every position · sped up: the real kernel moves 1 pixel at a time

Copyright © 2026 Nanyang Technological University. All rights reserved.

Convolutional Layers

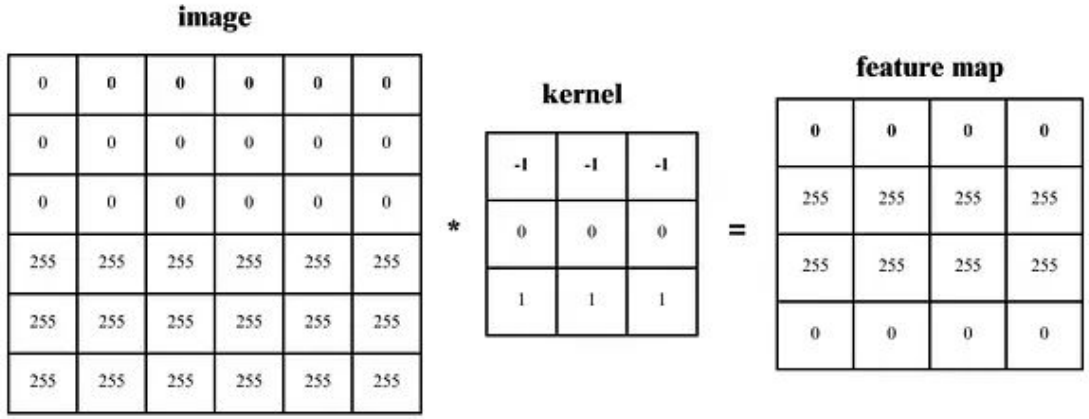


The convolutional layer is the first layer of a convolutional network.

- It takes the raw input image (pixels) and applies filters (tiny matrices) to detect simple patterns like edges, lines, or color gradients.
- Think of it like a visual scanner that's looking for very basic shapes.
- Each deeper layer learns more complex patterns by combining what previous layers saw.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Kernel Limitations



1x1 Kernel

Effectively just performs a linear combination of the input features at each spatial location.

It doesn't capture spatial relationships or local patterns because it only looks at a single pixel at a time.

2x2 Kernel

Captures a small, localized region of the input, but it's still too small to effectively capture complex patterns or high-level features.

It also introduces a lot of parameter redundancy and inefficiency for capturing larger spatial patterns.

4x4 Kernel

Captures a larger region of the image compared to 1x1 or 2x2 kernels.

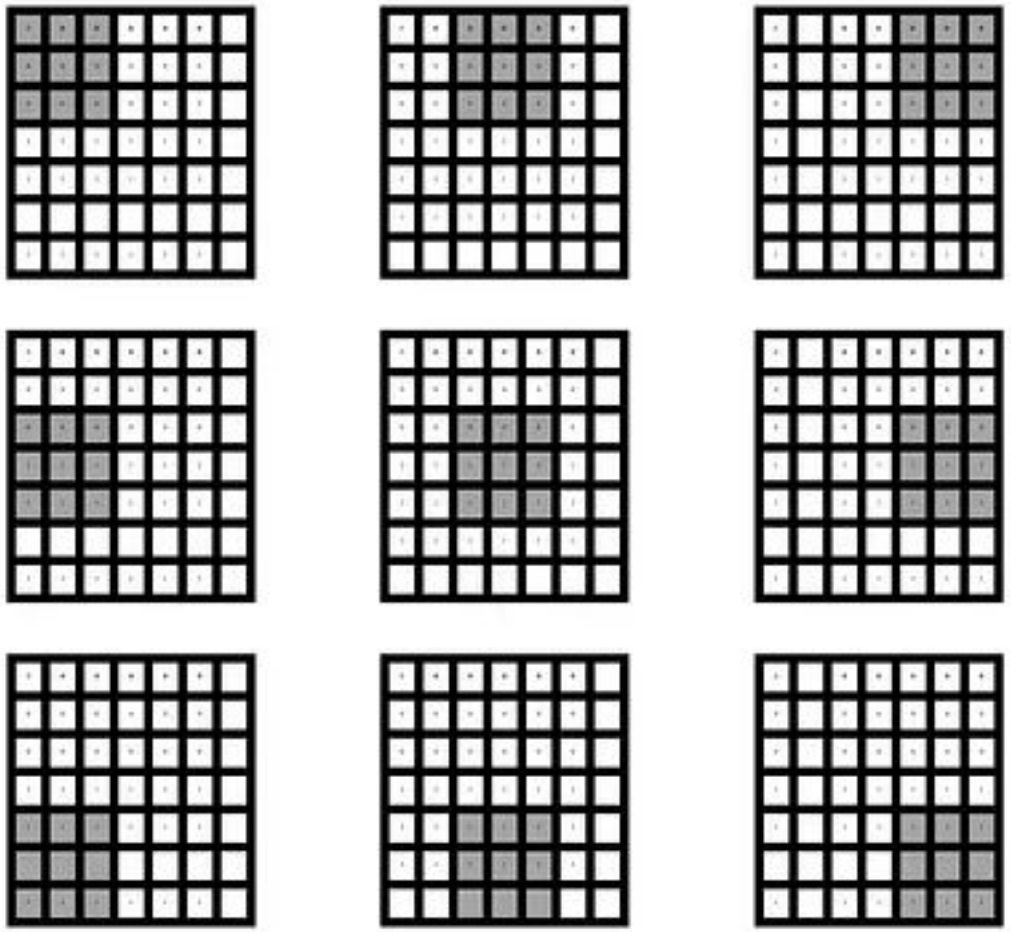
However, 2x2 and 4x4 are generally not preferred because odd-sized filters symmetrically divide the previous layer pixels around the output pixel.

If this symmetry is not present, there will be distortions across the layers which happens when using an even sized kernels

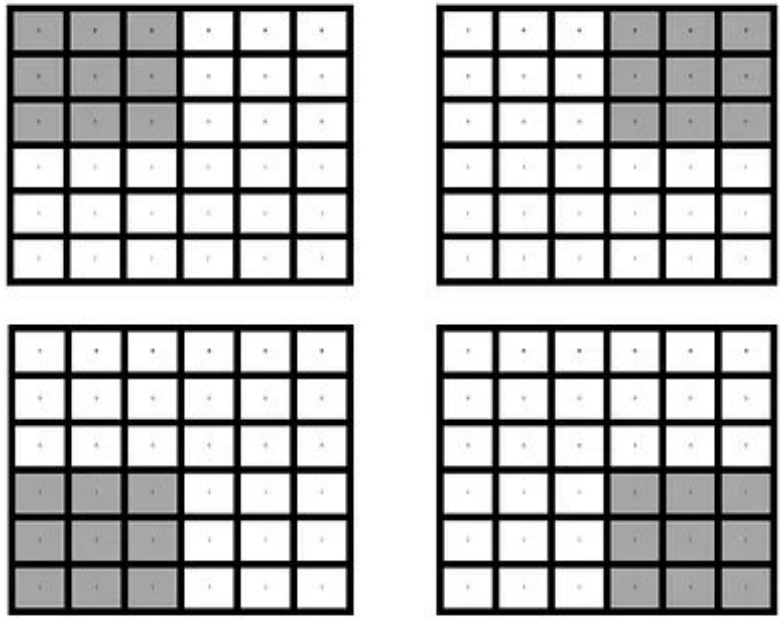
https://medium.com/codex/kernels-filters-in-convolutional-neural-network-cnn-lets-talk-about-them-ee4e94f3319

Copyright © 2026 Nanyang Technological University. All rights reserved.

Stride



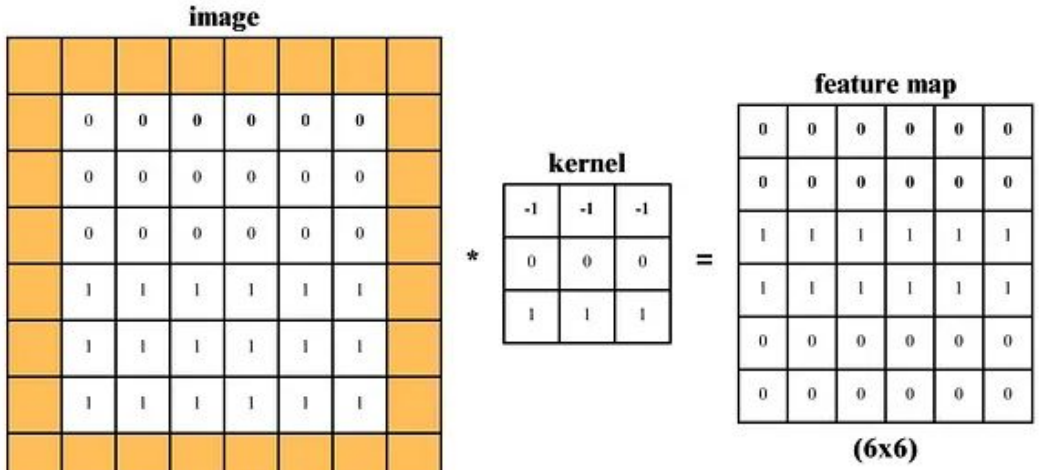
Stride = 2



Stride = 3

Copyright © 2026 Nanyang Technological University. All rights reserved.

Padding



1. **Preserving Spatial Dimensions:** Padding ensures that the spatial dimensions (height and width) of the input feature maps are preserved after applying convolutions. Without padding, the spatial dimensions would shrink after each convolutional operation. This can lead to a loss of information, especially at the edges of the input.

2. **Edge Information:** Padding helps preserve information at the borders or edges of the input image. When the convolutional kernel is applied near the boundaries, some parts of the kernel might not have access to data outside the image, so padding fills those gaps.

3. **Control over Output Size:** Padding allows control over the size of the output feature maps. Without padding, the output size decreases with each convolution, but by adding padding, you can ensure the output size is either the same as the input or controlled to a specific dimension.

https://medium.com/@minhazc.engg/padding-and-strides-in-cnn-58dc56493887

Copyright © 2026 Nanyang Technological University. All rights reserved.

Pooling Layers

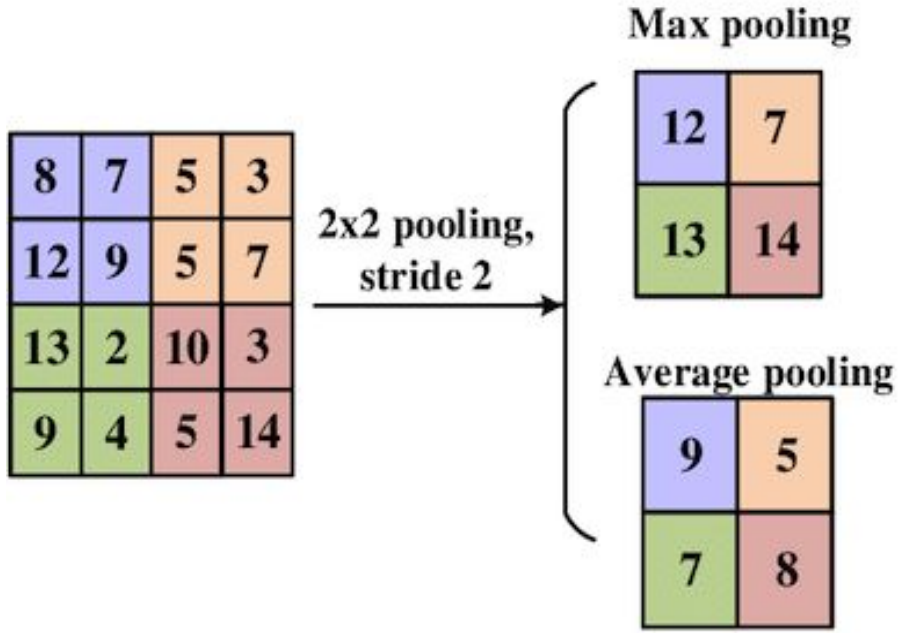
Pooling layers play a crucial role in convolutional neural networks by reducing the spatial size of the representation and the number of parameters, which helps control overfitting. They work by summarizing the features present in regions of the feature map generated by convolutional layers.

Max Pooling

Max pooling is the most common pooling operation, and it involves sliding a window (usually 2x2 or 3x3 in size) over the feature map and taking the maximum value within the window as the output.

Average Pooling

Average pooling takes the average of the elements within the window. It is less common than max pooling

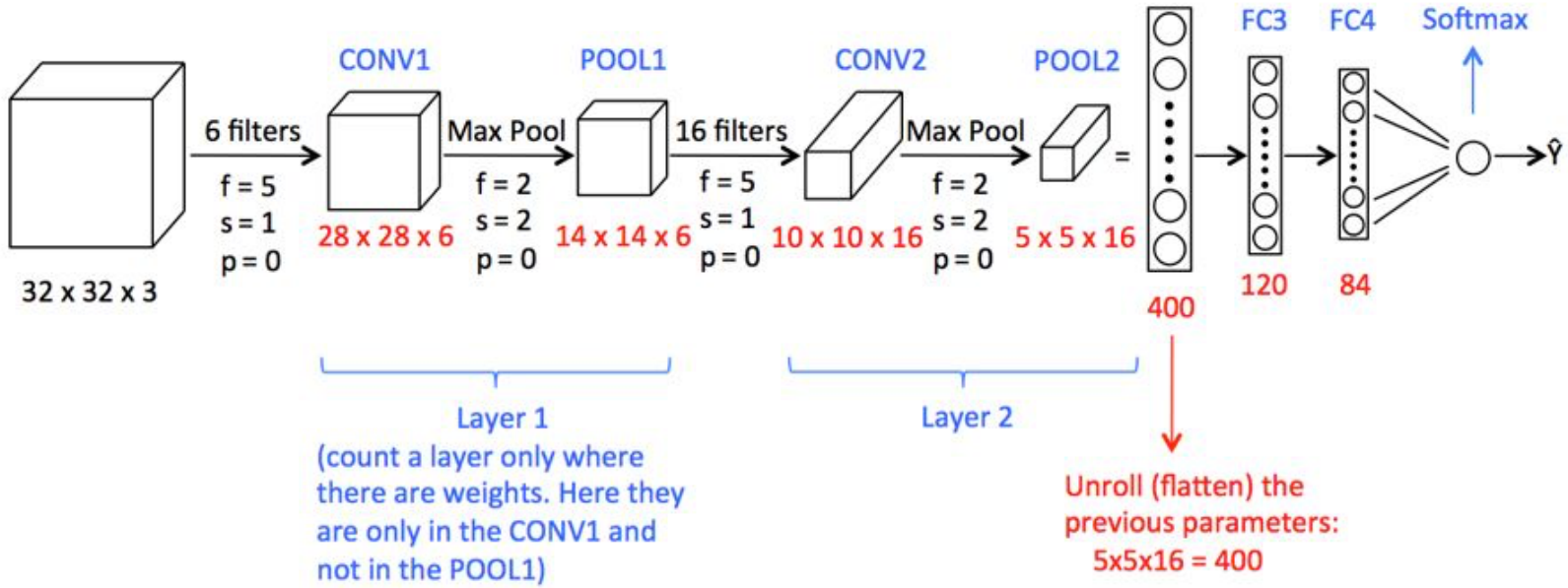


Copyright © 2026 Nanyang Technological University. All rights reserved.

Spatial Calculation

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

n_{in}: number of input features
n_{out}: number of output features
k: convolution kernel size
p: convolution padding size
s: convolution stride size



Copyright © 2026 Nanyang Technological University. All rights reserved.

The CNN Block

Conv → ReLU → MaxPool

Three operations, stacked 2-3 times, then a Flatten + Linear head

```
Conv2d(in, out, kernel=3, padding=1)
        ↓
      ReLU()
        ↓
   MaxPool2d(2)
```

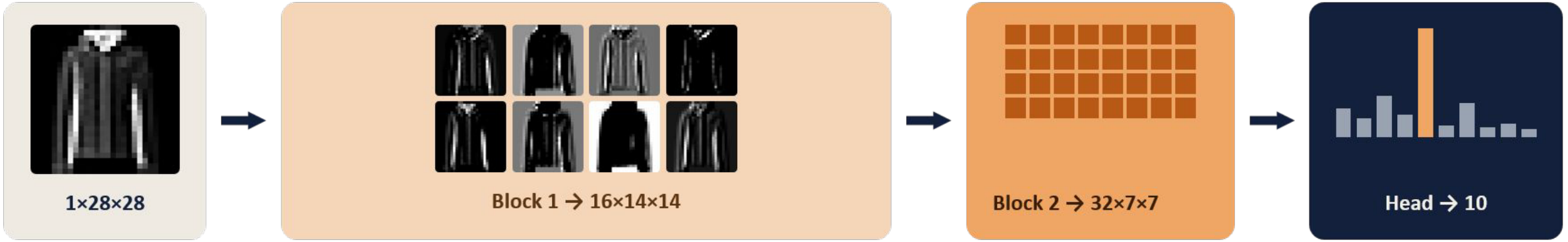
Repeat 2-3 times, doubling output channels each block.

Final spatial size shrinks (28→14→7), channel count grows (1→16→32). Then Flatten + Linear.

The L07 training loop is unchanged — only the model definition changes. This TinyCNN (~106K params) beats the 235K-param flat MLP: 0.884 vs 0.871 test accuracy.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Two conv blocks and a classifier head




```
nn.Conv2d(1, 16, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
nn.Flatten(), nn.Linear(32*7*7, 64), nn.ReLU(), nn.Linear(64, 10)
```

Copyright © 2026 Nanyang Technological University. All rights reserved.

The CNN Block

Conv → ReLU → MaxPool

Input

1×28×28

1 gray channel

→

Conv 1→16

16×28×28

16 kernels; padding 1 keeps 28

→

MaxPool 2

16×14×14

28 ÷ 2 = 14

→

Conv 16→32

32×14×14

32 kernels; size kept

→

MaxPool 2

32×7×7

14 ÷ 2 = 7

→

Flatten

1,568

32 × 7 × 7 in one long row

1,568 inputs × 64 outputs = 100,352 weights

Every input connects to every output. Add 64 biases: **100,416**, which is **95%** of TinyCNN's 105,866 parameters.

Conv 1: 1×16×3×3 + 16 = **160**

Conv 2: 16×32×3×3 + 32 = **4,640**

Linear 1: 1,568×64 + 64 = **100,416**

Linear 2: 64×10 + 10 = **650**

Copyright © 2026 Nanyang Technological University. All rights reserved.

ImageNet: computer vision's Olympics

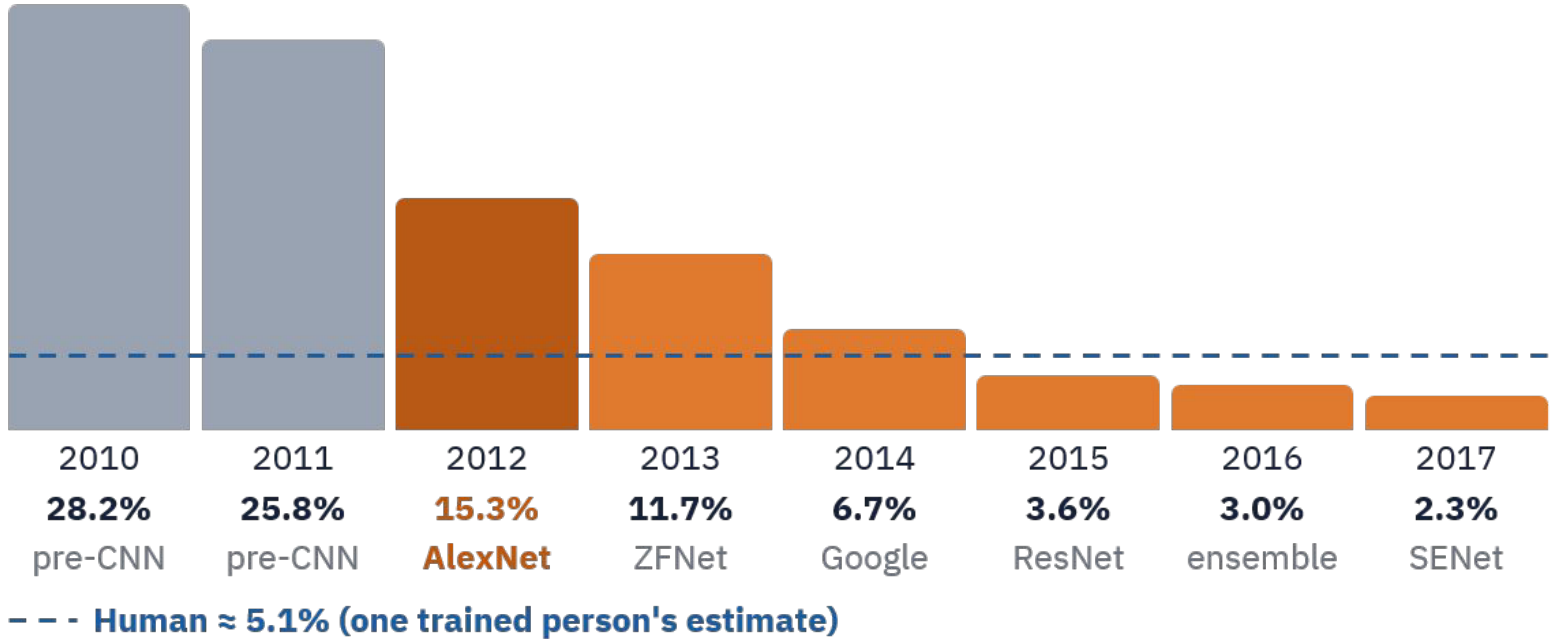
1.2M training photos, 1,000 classes, from 120 dog breeds to mushrooms and fire trucks.

A yearly contest, 2010–2017 (ILSVRC): teams labelled 100,000 hidden test photos.

Score = top-5 error: the true label is missing from the model's 5 best guesses.

2012: AlexNet, a CNN trained on 2 gaming GPUs, cut error from 26% to 15%. Deep learning took off.

Winning top-5 error by year (lower is better)



The contest ended in 2017

Top-5 error had fallen to 2.3%, below the ~5% human estimate.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Famous CNN architectures

Model	Layers	First conv	Kernels and pooling	Weights	Big idea
LeNet-5 (1998)	5	5×5, stride 1	5×5 · average pool 2×2	60K	First working CNN
AlexNet (2012)	8	11×11, stride 4	11, 5, 3 · max pool 3×3/2	60M	ReLU + GPUs win ImageNet
VGG-16 (2014)	16	3×3, stride 1	only 3×3 · max pool 2×2	138M	Small kernels, stacked deep
GoogLeNet (2014)	22	7×7, stride 2	1×1, 3×3, 5×5 side by side	7M	Mix kernel sizes at once
ResNet-18 (2015)	18	7×7, stride 2	3×3 · global average pool	11.7M	Skip connections allow depth
TinyCNN (ours)	4	3×3, stride 1	3×3 · max pool 2×2	106K	Fashion-MNIST, 28×28

Copyright © 2026 Nanyang Technological University. All rights reserved.

Hire a beginner, or retrain an experienced artist?



From scratch

Teach someone who has never seen an image: edges, curves, shadows, fabric, then shirts versus coats. With 2,000 photos they memorise instead of learn.



Transfer learning

Hire someone who has studied a million photos. They already know edges and textures; you only teach the final job. That needs far fewer examples.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Transfer Learning — Why

Real catalogues have hundreds of images, not millions

The reality at NorthStar

Fashion-MNIST has 60,000 training images. Marcus's catalogue has ~500 per class.
Train a CNN from scratch on 500 images and it OVERFITS — memorises noise.

The fix — don't start from scratch

Start from a model someone else trained on a much bigger dataset (ImageNet, 1.2M photos).
Keep its hard-won feature detectors. Replace only the final classification head.
Train that head on YOUR 500 images.

You're not training a model — you're adapting one.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Two Transfer-Learning Patterns

Feature extraction vs fine-tuning

FEATURE EXTRACTION

1. Freeze every layer
2. Replace classifier head
3. Train only the head

Fast. Few trainable params. Works when target domain matches pretraining domain.

FINE-TUNING

1. Freeze most of the backbone
2. Unfreeze the LAST conv block (layer4)
3. Train with a TINY learning rate (1e-4)

Slower. More params updated. Adapts pretrained features when domain gap is large.

Copyright © 2026 Nanyang Technological University. All rights reserved.

The Honest Result

Transfer learning isn't free — domain match matters

ImageNet (colour photos) → Fashion-MNIST (grayscale silhouettes): big domain gap

Approach (2K imgs, head-only or fine-tune)	Test accuracy
TinyCNN from scratch	`0.787`
ResNet18 transfer — head only	`0.715` ← loses!
ResNet18 — fine-tune layer4	`0.831` ← wins

Lesson: transfer learning isn't magic. On a wide domain gap, head-only feature extraction can lose. Fine-tune the last conv block at a tiny LR to recover.
For Marcus's real catalogue (colour photos, closer to ImageNet), expect a much smaller gap.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Benefits of Pre-trained models

Pre-trained models are trained on large benchmark datasets, such as ImageNet, which has over a million images with 1000 categories. Using these models can lead to the following benefits:

1. **Time-saving:** Training a model from scratch requires a significant amount of data and computational resources. Pre-trained models have already learned a lot of features, which can drastically reduce the training time.
2. **Improved performance:** Pre-trained models often increase the performance of neural networks, especially when the amount of training data for the new task is limited.
3. **Feature extraction:** Models pre-trained on large datasets have learned to recognize a variety of features that can be useful for new tasks, even if those tasks involve different classes or categories.

Copyright © 2026 Nanyang Technological University. All rights reserved.

When Transfer Learning may not be ideal

Transfer learning is a powerful technique in deep learning, where you use a pre-trained model on a large dataset and fine-tune it for your specific task. However, there are situations where transfer learning might not be ideal.

Here are some scenarios where transfer learning might not be the best option:

1. Your Task and Dataset Are Very Different from the Pre-trained Model's Task:

- **Problem:** Transfer learning works best when the pre-trained model was trained on a dataset that is similar to your dataset in terms of domain, content, and characteristics. If the pre-trained model was trained on a completely different type of data (e.g., a model trained on natural images like ImageNet being used on medical images), transfer learning may not perform well.
- **Example:** Using a model trained on ImageNet for classifying medical scans (like X-rays or MRI images) might not yield good results because the feature representations learned from natural images may not be applicable to medical images.

Copyright © 2026 Nanyang Technological University. All rights reserved.

When Transfer Learning may not be ideal

2. Insufficient Data for Fine-tuning:

- **Problem:** Transfer learning usually requires some amount of fine-tuning with your specific data. If you have very little data for your task, fine-tuning might lead to overfitting, as the pre-trained model's weights might not generalize well to the small dataset.
- **Example:** If you're trying to fine-tune a model with a few hundred images, the model may not adapt properly and could perform worse than training from scratch.

3. When the Task Requires Domain-Specific Knowledge:

- **Problem:** If your task requires very specialized domain knowledge that the pre-trained model hasn’t encountered, transfer learning might not work well. Models trained on general datasets (like ImageNet) might not capture domain-specific features that are crucial for your task.
- **Example:** Using a general pre-trained model for natural language processing tasks on a specific legal document corpus might not work well because the pre-trained model hasn't been exposed to the unique vocabulary and structures of legal texts.

Copyright © 2026 Nanyang Technological University. All rights reserved.

CODE-ALONG

Open: 04_transfer_learning.ipynb

What you'll do

- Subsample Fashion-MNIST to 200 images per class (small data)
- Train TinyCNN from scratch as baseline
- Load pretrained ResNet18, freeze backbone, train head only
- Sweep training set size, then fine-tune layer4 in Extension

25 minutes · Stop at the Extension marker

Sarah's Recommendation

Production-ready transfer learning

Sarah, end of L08:

"Start with a pretrained ResNet18. Train new head for a few epochs.
Unfreeze the last conv block (layer4) at LR 1e-4. Fine-tune until val flattens.
Expect to land in the low-90s top-1 on the 10 main categories with 500 photos per class — projected, since this colour catalogue sits much closer to ImageNet than Fashion-MNIST."

Recipe

1. Load pretrained ResNet18 backbone
2. Replace fc head → 10-class linear layer
3. Train head for 3 epochs at LR 1e-3
4. Unfreeze layer4, fine-tune at LR 1e-4 until validation curve flattens

Copyright © 2026 Nanyang Technological University. All rights reserved.

Three Things To Remember

L08 takeaways

1

Convolutions exploit locality and translation invariance

Same kernel slides everywhere → fewer params, better inductive bias for images.

2

The PyTorch training loop doesn't change

Conv2d / MaxPool2d are drop-in replacements for Linear. L07's loop just works.

3

Don't train CNNs from scratch on small data

Fine-tune a pretrained backbone (0.831 vs 0.787 from scratch). Head-only can lose on a domain gap.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Bridge to L09

From pixels to words

Marcus, after the auto-tagger ships:

"Can we also do natural-language search?
A customer types 'blue summer dress' — surface the right products."

Next modality: text

L09 — Embeddings & NLP foundations	Turning words into vectors. Tokenisation, word embeddings, similarity search.
L10 — Transformers & GenAI	The architecture behind GPT, BERT, ChatGPT. Attention, generation, fine-tuning.

Copyright © 2026 Nanyang Technological University. All rights reserved.